

GUVI HCL
FULLSTACK DATA SCIENCE PROGRAM

Job Acceptance Prediction System

Technical & User Documentation

50,000

Candidate Records

5

ML Algorithms

0.698

Best Model AUC-ROC

20

Visualizations

Domain: HR Analytics & Predictive Modeling
Tech Stack: Python · Pandas · Scikit-learn · Streamlit · Plotly
Version 1.0

Table of Contents

1. Executive Summary

The Job Acceptance Prediction System is an end-to-end machine learning pipeline that predicts whether a job candidate will be placed (hired) or not placed, based on academic performance, interview outcomes, skills alignment, and market conditions. The system was built for the GUVI HCL Fullstack Data Science with Generative & Agentic AI Program and demonstrates a complete data science workflow — from raw, messy data to a live, interactive prediction dashboard.

The project processes 50,000 synthetic candidate records through a rigorous six-step cleaning pipeline, engineers eight analytically meaningful features, generates 15 exploratory visualizations, trains and compares five machine learning algorithms, and deploys the best-performing model inside a four-page Streamlit web application with real-time prediction capability.

1.1 Key Outcomes

- Cleaned a 51,000-row raw dataset down to 50,000 verified unique candidate records, correcting 27,294 missing values and standardizing 17,754 inconsistently formatted category labels.
- Engineered 8 new features, including a composite Placement Probability score (0–100) and a High-Risk candidate flag.
- Produced 15 static exploratory data analysis charts and 5 model-evaluation visualizations (20 total), covering every major dimension of the dataset.
- Trained and benchmarked 5 classification algorithms — Logistic Regression, K-Nearest Neighbors, Decision Tree, Random Forest, and Gradient Boosting — using stratified 5-fold cross-validation.
- Selected Logistic Regression as the best-performing model, achieving an AUC-ROC of 0.698 on a held-out 10,000-record test set, with the most stable cross-validation performance of all candidates.
- Delivered a fully interactive, four-page Streamlit dashboard with live candidate-level prediction, gauge visualizations, and filterable analytics.

1.2 Intended Audience

This document is written for three audiences:

- Recruiters / HR analysts — who will use the live dashboard to score candidates and interpret placement-driving factors (see Section 8).
- Data science reviewers / evaluators — who need a full account of the methodology, metrics, and model-selection rationale (see Sections 4–7).
- Developers — who need to install, run, extend, or maintain the codebase (see Sections 9–11).

2. Problem Statement

Recruitment teams evaluate large volumes of candidates across multiple dimensions — academic records, interview performance, skill alignment, and market competitiveness — but often lack a systematic, data-driven way to prioritize outreach or identify candidates who are likely to be overlooked despite strong underlying potential.

2.1 Objective

Build a supervised binary classification system that predicts a candidate's placement outcome (Placed vs Not Placed) from their profile, and expose that prediction through an interactive tool that recruiters can use directly — without needing to run code or interpret raw model output.

2.2 Success Criteria

- A complete, reproducible data cleaning pipeline that handles realistic data quality issues (missing values, inconsistent categories, duplicates).
- At least five machine learning algorithms trained and objectively compared on the same train/test split.
- A single best model selected using a defensible, threshold-independent metric (AUC-ROC).
- A working dashboard that turns a candidate's raw attributes into an acceptance probability and an interpretable risk signal in real time.

2.3 Scope & Assumptions

- The dataset used is synthetically generated to mirror realistic HR data distributions and deliberately includes data quality defects for the cleaning pipeline to resolve — it is not sourced from a real company's records.
- The model is a decision-support tool, not an autonomous hiring decision-maker. Final outcomes should always involve human judgment.
- Feature relationships (e.g., interview score correlating with placement) are calibrated to be directionally realistic but are simulated, not empirically observed.

3. Dataset Documentation

3.1 Overview

Property	Value
Total records (raw)	51,000
Total records (after cleaning)	50,000
Raw columns	23
Columns after feature engineering	39
Target variable	status (Placed / Not Placed)
Class balance	56.3% Placed / 43.7% Not Placed
Source file (raw)	data/job_placement_raw.csv
Source file (cleaned)	outputs/job_placement_cleaned.csv

3.2 Raw Column Reference

The raw dataset generated by data/generator.py contains 23 columns, grouped into six logical feature families:

Identifiers

- candidate_id — unique candidate identifier (e.g., CAND000001)

Demographic

- age — candidate age, 20–35
- gender — Male / Female / Other (with dirty variants, e.g. 'male', 'MALE', 'M')
- degree — B.Tech, BCA, BSc, MCA, MBA, B.Com

Academic

- tenth_percentage — Class 10 exam score (%)
- twelfth_percentage — Class 12 exam score (%)
- gpa — cumulative GPA, scale 4.0–10.0
- backlogs — number of academic backlogs, 0–4

Experience & Skills

- years_experience — professional experience in years
- num_skills — count of listed technical skills, 1–9
- skills_match_pct — % match between candidate skills and job requirements

- `certification` — professional certification held (AWS, Google Cloud, Azure, PMP, Salesforce, Coursera ML, or None)
- `projects_done` — number of completed projects, 0–8

Assessment

- `num_interviews` — number of interview rounds attended
- `interview_score` — interview performance score, 20–100
- `aptitude_score` — aptitude test score, 10–100
- `employability_score` — composite employability rating, 10–100

Market / Job Context

- `company_tier` — Tier 1 / Tier 2 / Tier 3 (with dirty variants, e.g. 'tier1', 'TIER 1')
- `job_role` — one of 10 roles (Software Engineer, Data Analyst, ML Engineer, Product Manager, Business Analyst, DevOps Engineer, Frontend Developer, Backend Developer, Data Scientist, QA Engineer)
- `offered_salary_inr` — offered annual salary in Indian Rupees
- `competition_level` — Low / Medium / High (with dirty variants)
- `location_pref_match` — binary flag: does job location match candidate preference?

Target

- `status` — Placed or Not Placed (the label to be predicted)

3.3 Intentional Data Quality Issues

To simulate a realistic, messy source system, the data generator deliberately injects the following defects, which the cleaning pipeline (Section 4) is responsible for resolving:

Issue Type	Affected Columns	Injection Rate
Missing values (nulls)	<code>gpa</code> , <code>interview_score</code> , <code>skills_match_pct</code> , <code>employability_score</code> , <code>years_experience</code> , <code>aptitude_score</code> , <code>offered_salary_inr</code> , <code>certification</code>	5% – 10% per column
Inconsistent category labels	<code>company_tier</code> , <code>competition_level</code> , <code>gender</code>	~12% of values per column
Duplicate-like records	All columns (row-level)	~2% of rows

Example inconsistent label variants injected for `company_tier`: "Tier 1", "tier1", "TIER1", "tier 1", "Tier-1", "t1". The cleaning pipeline maps all of these back to a single canonical value.

4. Data Cleaning Pipeline

Implemented in preprocessing/pipeline.py, the cleaning pipeline runs as six sequential, independently testable steps. Every step logs a machine-readable report to outputs/reports/cleaning_report.txt.

4.1 Step-by-Step Reference

Step	Name	Method	Result (actual run)
S1	Remove Duplicates	Composite-key match on GPA + scores + tier + status, keep first occurrence	1,000 duplicate rows removed (51,000 → 50,000)
S2	Standardize Categoricals	Canonical dictionary mapping for company_tier, competition_level, gender	17,754 values corrected across 3 columns
S3	Handle Missing Values	Median imputation (numeric), mode imputation (categorical)	27,294 cells imputed
S4	Consistency Checks	Clip out-of-range scores to valid bounds (e.g., GPA within 4.0–10.0)	Range validation pass
S5	Encode Target	Map status → status_num (Placed = 1, Not Placed = 0)	Placement rate: 56.26%
S6	Encode & Scale	Label-encode all categorical features; StandardScaler on numeric features	ML-ready feature matrix produced

4.2 Category Standardization Logic

Step S2 uses explicit canonical mapping dictionaries rather than fuzzy matching, to guarantee deterministic, auditable results. Example mapping for company_tier:

```
TIER_MAP = {
    'tier1': 'Tier 1', 'tier 1': 'Tier 1', 'tier-1': 'Tier 1',
    't1': 'Tier 1', 'TIER1': 'Tier 1', 'TIER 1': 'Tier 1',
    'tier2': 'Tier 2', ... , 'tier3': 'Tier 3', ...
}
```

4.3 Missing Value Imputation Detail

Actual imputation counts and fill values from the production run, as recorded in outputs/reports/cleaning_report.txt:

Column	Method	Cells Filled	Fill Value
gpa	Median	2,924	7.19
years_experience	Median	2,431	1.6

Column	Method	Cells Filled	Fill Value
skills_match_pct	Median	3,494	60.8
interview_score	Median	3,983	63.0
aptitude_score	Median	4,973	57.0
employability_score	Median	4,479	58.9
offered_salary_inr	Median	2,021	₹902,670
certification	Mode	2,989	None

Why median/mode, not mean?

Median and mode imputation are robust to outliers and skewed distributions, which is important here because scores and salaries are not normally distributed. Using the mean would let a small number of extreme values distort the fill value for thousands of missing cells.

5. Feature Engineering

Implemented in `features/engineering.py`, this stage derives eight new columns that translate raw scores into business-meaningful signals. These features are engineered after cleaning but before model training, and are also surfaced directly in the dashboard.

5.1 Derived Feature Reference

Feature	Type	Definition
<code>experience_category</code>	Categorical (4 bands)	Fresher (0 yrs) / Junior (0–2) / Mid-Level (2–5) / Senior (5+)
<code>academic_score</code>	Numeric (0–100)	$0.25 \times 10\text{th\%} + 0.25 \times 12\text{th\%} + 0.50 \times (\text{GPA} \times 10)$
<code>academic_band</code>	Categorical (5 bands)	Poor / Below Avg / Average / Good / Excellent, from <code>academic_score</code>
<code>skills_level</code>	Categorical (4 bands)	Low / Medium / High / Excellent, from <code>skills_match_pct</code>
<code>interview_category</code>	Categorical (5 bands)	Very Poor / Poor / Average / Good / Excellent, from <code>interview_score</code>
<code>has_certification</code>	Binary (0/1)	1 if certification is not "None", else 0
<code>placement_probability</code>	Numeric (0–100)	Weighted composite score — see formula below
<code>high_risk</code>	Binary (0/1)	1 if <code>placement_probability</code> < 45, OR <code>interview_score</code> < 40, OR <code>backlogs</code> ≥ 2

5.2 Placement Probability Formula

The `placement_probability` feature is the system's key business-facing metric — a single 0–100 score summarizing a candidate's overall strength. It is computed as a weighted blend of the four assessment dimensions shown to correlate most strongly with placement outcome in the correlation analysis (Section 6.4):

```
placement_probability =
    interview_score           x 0.30
  + skills_match_pct         x 0.25
  + academic_score           x 0.20
  + employability_score      x 0.15
  + has_certification x 10    x 0.10
  - backlogs x 3
(cropped to the range 0-100)
```

5.3 High-Risk Flag

The `high_risk` flag exists to make the model's output directly actionable for recruiters. Rather than requiring a user to interpret a raw probability number, the dashboard surfaces a binary risk badge (Low / High) so at-risk candidates can be proactively identified for additional support before a final decision is made. A candidate is flagged high-risk if any of the following are true:

- Placement probability score is below 45
- Interview score is below 40
- The candidate has 2 or more academic backlogs

6. Exploratory Data Analysis

Implemented in `eda/visualizations.py`, this stage produces 15 static charts (Matplotlib/Seaborn, saved as PNG to `outputs/eda_charts/`) that examine every major dimension of the cleaned dataset against the placement outcome. All charts use a consistent dark HR-analytics color theme matching the dashboard.

6.1 Chart Index

#	Chart	What It Shows
01	Placement Overview	Overall Placed / Not Placed split; acceptance rate by company tier
02	Academic vs Placement	GPA, 10th %, and 12th % distributions split by outcome
03	Interview vs Placement	Interview and employability score distributions with medians
04	Skills Match Analysis	Boxplot of skills match %; placement counts by skills level
05	Company Tier Analysis	Acceptance rate and candidate volume by tier
06	Experience Analysis	Years-experience histogram; placement rate by experience category
07	Competition Level	Acceptance rate and volume by Low/Medium/High competition
08	Certification Impact	Acceptance rate by certification type; certified vs non-certified
09	Correlation Heatmap	Pearson correlation matrix across all numeric features
10	Placement Probability	Distribution and violin plot of the composite probability score
11	Backlogs Impact	Placement rate as a function of backlog count
12	Salary Analysis	Offered-salary distribution; acceptance rate by salary band
13	Gender Analysis	Placement rate and candidate distribution by gender
14	Job Role Analysis	Placement rate and application volume by job role
15	Academic Band	Placement rate across the 5-tier academic band

6.2 Headline Findings

- Overall placement rate is 56.3% — a reasonably balanced target for binary classification, avoiding the need for heavy class-imbalance correction.
- Tier 1 companies show the highest acceptance rate, consistent with more selective, better-prepared applicant pools at top-tier firms.
- Interview score and skills-match percentage show the strongest positive correlation with placement outcome of any numeric feature.
- Backlogs show a clear negative relationship with placement — each additional backlog measurably reduces acceptance likelihood.

- Certified candidates show a meaningfully higher acceptance rate than uncertified peers across nearly every certification type.

6.3 Chart Design Standards

- Consistent color coding throughout: green (#06D6A0) = Placed, rose (#EF476F) = Not Placed.
- Every distribution chart includes median reference lines for both outcome classes.
- Every categorical breakdown chart pairs a volume view with a rate view, so raw counts are never presented without context.

7. Model Training & Evaluation

Implemented in `models/training.py`, this is the core predictive-modeling stage. Five classification algorithms are trained on an identical feature set and train/test split, then compared using six evaluation metrics.

7.1 Training Configuration

Parameter	Value
Train / test split	80% / 20%, stratified by target
Training samples	40,000
Test samples	10,000
Features used	25 (numeric + label-encoded categorical)
Cross-validation	5-fold Stratified K-Fold
Class distribution	Placed 56.3% (28,131) / Not Placed 43.7% (21,869)
Primary selection metric	AUC-ROC on held-out test set
Random seed	42 (fixed for reproducibility)

7.2 Algorithms Compared

Algorithm	Family	Key Hyperparameters
Logistic Regression	Linear	$C = 0.8$, <code>class_weight = balanced</code> , <code>max_iter = 2000</code>
K-Nearest Neighbors	Instance-based	$k = 7$, <code>weights = distance</code>
Decision Tree	Tree-based	<code>max_depth = 8</code> , <code>min_samples_leaf = 10</code> , <code>balanced</code>
Random Forest	Ensemble (Bagging)	150 trees, <code>max_depth = 10</code> , <code>balanced</code>
Gradient Boosting	Ensemble (Boosting)	100 estimators, <code>learning_rate = 0.1</code> , <code>max_depth = 4</code>

7.3 Full Results

Model	Accuracy	Precision	Recall	F1	AUC-ROC	CV-AUC (5-fold)
Logistic Regression * BEST	63.9%	69.7%	63.5%	0.665	0.698	0.697 ± 0.004
Gradient Boosting	64.7%	66.3%	75.8%	0.707	0.696	0.692 ± 0.004
Random Forest	63.8%	68.7%	65.5%	0.670	0.689	0.689 ± 0.003

Model	Accuracy	Precision	Recall	F1	AUC-ROC	CV-AUC (5-fold)
Decision Tree	60.8%	68.0%	57.1%	0.621	0.656	0.657 ± 0.003
K-Nearest Neighbors	58.0%	61.5%	67.8%	0.645	0.596	0.592 ± 0.004

7.4 Model Selection Rationale

Logistic Regression was selected as the best model on the basis of AUC-ROC — the primary metric defined in the training pipeline — where it achieved the top score (0.698) and, notably, the tightest cross-validation spread (± 0.004) of all five candidates, indicating the most consistent generalization across folds.

AUC-ROC was chosen as the primary selection metric for three reasons:

- It is threshold-independent, measuring how well a model ranks Placed candidates above Not Placed candidates across every possible decision cutoff — appropriate for a scoring tool rather than a fixed-threshold classifier.
- It is robust to the moderate class imbalance present in this dataset (56% / 44%).
- It allows direct, fair comparison across fundamentally different algorithm families (linear, distance-based, tree-based, ensemble).

Worth noting

Gradient Boosting achieved the highest F1 score (0.707), driven by strong recall (75.8%). Teams that prioritize catching every possible Placed candidate over minimizing false positives may prefer Gradient Boosting in production, despite Logistic Regression's higher AUC-ROC. This tradeoff should be revisited if the business objective shifts.

7.5 Feature Importance

For the selected Logistic Regression model, feature importance is derived from the absolute value of the standardized coefficients. Consistent with the correlation analysis in Section 6, `interview_score`, `skills_match_pct`, and `academic_score` are the three most influential predictors.

7.6 Evaluation Artifacts

The following files are generated automatically at the end of training and saved to the `outputs/` directory:

- `outputs/eda_charts/16_roc_curves.png` — ROC curve overlay for all 5 models
- `outputs/eda_charts/17_model_comparison.png` — grouped bar chart across 6 metrics
- `outputs/eda_charts/18_feature_importance.png` — top-15 feature importance for the best model
- `outputs/eda_charts/19_confusion_matrix.png` — confusion matrix for the best model on the test set
- `outputs/eda_charts/20_algorithm_ranking.png` — horizontal bar ranking all models by AUC-ROC
- `outputs/reports/model_evaluation.txt` — full text report with per-model classification reports
- `outputs/model_plots/*.pkl` — every trained model serialized with joblib, plus `best_model.pkl` and `feature_names.pkl`

8. Dashboard User Guide

The Streamlit dashboard (dashboard/app.py) is the primary interface for non-technical users. It loads the cleaned dataset and trained models once (using `@st.cache_resource` for performance) and exposes four pages via a sidebar navigation.

8.1 Global Filters

Available on every page via the sidebar:

- Company Tier — All / Tier 1 / Tier 2 / Tier 3
- Competition Level — All / Low / Medium / High
- Experience Category — All / Fresher / Junior / Mid-Level / Senior
- Status — All / Placed / Not Placed

8.2 Page 1 — Overview

The landing page presents seven live KPI cards computed from the currently filtered dataset:

KPI	Definition
Total Candidates	Count of records matching current filters
Placement Rate	% of filtered candidates with status = Placed
Offer Dropout Rate	100% - Placement Rate
Avg Interview Score	Mean interview_score across filtered candidates
Avg Skills Match	Mean skills_match_pct across filtered candidates
High-Risk Candidates	% of filtered candidates with high_risk = 1
Avg Placement Score	Mean placement_probability across filtered candidates

Below the KPI row: a placement-split donut chart, a placement-rate-by-tier bar chart, an experience-category breakdown, a competition-level impact chart, and a top-job-roles chart — all rendered with Plotly for interactivity (hover, zoom, and export).

8.3 Page 2 — EDA Charts

Reproduces the 15 static EDA analyses as interactive Plotly visuals, organized into four tabs:

- Academic — GPA and 10th/12th % distributions, academic band vs. placement rate
- Interview & Skills — score distributions, skills-match boxplot, aptitude vs. interview scatter
- Certifications — acceptance rate by certification type, certified vs. non-certified comparison
- Correlations — full numeric correlation heatmap, placement probability violin plot

8.4 Page 3 — Model Results

Displays the five model-evaluation charts (ROC curves, metrics comparison, feature importance, confusion matrix, algorithm ranking) generated during training, plus a live-parsed summary table of all model metrics with the best model highlighted.

8.5 Page 4 — Predict

The dashboard's flagship feature: a live scoring form. The user enters a candidate's profile across three columns (Academic, Experience & Skills, Interview & Job) and clicks Predict Acceptance. The system then:

1. Recomputes all engineered features (academic score, placement probability, risk flag, etc.) from the entered values, using the same logic as the training pipeline.
2. Encodes categorical inputs to match the exact feature schema the model was trained on (25 features, loaded from `outputs/model_plots/feature_names.pkl`).
3. Runs the saved best model (`outputs/model_plots/best_model.pkl`) to obtain an acceptance probability.
4. Displays a Placed / Not Placed verdict, the probability as a percentage, a Plotly gauge chart, a Low/High risk badge, and a bar breakdown of the key contributing factors.

Design note

The Predict page intentionally re-implements the feature engineering logic inline (rather than importing shared functions) so a single-row prediction does not depend on the bulk dataset being loaded — this keeps the page responsive even on first load.

9. System Architecture

9.1 Project Structure

```

job_acceptance_prediction_system/
|-- setup.py           One-click pipeline runner
|-- requirements.txt    Python dependencies
|-- README.md          Quick-start guide
|-- config/
|   +-- settings.py     Paths, colors, constants
|-- data/
|   +-- generator.py     Synthetic dataset generator
|-- preprocessing/
|   +-- pipeline.py      6-step cleaning pipeline
|-- features/
|   +-- engineering.py   8 derived features
|-- eda/
|   +-- visualizations.py 15 EDA charts
|-- models/
|   +-- training.py      5-algorithm training & selection
|-- dashboard/
|   +-- app.py           4-page Streamlit application
+-- outputs/
|   |-- eda_charts/      20 PNG visualizations
|   |-- model_plots/     Trained .pkl model files
|   |-- reports/         Cleaning & evaluation reports
|   +-- job_placement_cleaned.csv Final cleaned dataset

```

9.2 Data Flow

The pipeline executes as a strict linear sequence, with each stage reading the output of the previous one:

5. data/generator.py creates the raw synthetic dataset → data/job_placement_raw.csv
6. preprocessing/pipeline.py cleans it → outputs/job_placement_cleaned.csv, plus scaler.pkl and encoders.pkl
7. features/engineering.py adds derived columns to the in-memory DataFrame
8. eda/visualizations.py reads the engineered DataFrame → 15 PNG files in outputs/eda_charts/
9. models/training.py reads the same DataFrame → 5 trained models + 5 evaluation charts + text report
10. dashboard/app.py loads the cleaned CSV and best_model.pkl at runtime to serve the live application

9.3 Technology Stack

Layer	Technology	Purpose
Language	Python 3.9+	Core implementation language
Data handling	Pandas, NumPy	DataFrame operations, numeric computation
Static visualization	Matplotlib, Seaborn	15 EDA charts + evaluation plots
Interactive visualization	Plotly	All dashboard charts and gauges
Machine learning	Scikit-learn	5 classification algorithms, metrics, CV
Web application	Streamlit	4-page interactive dashboard
Model persistence	Joblib	Serializing trained models to .pkl
Database readiness	SQLAlchemy	Included for future DB-backed deployment

10. Installation & Setup Guide

10.1 Prerequisites

- Python 3.9 or later
- pip package manager
- ~200 MB free disk space (for generated charts, models, and datasets)

10.2 Step-by-Step Installation

Step 1 — Install dependencies

```
pip install -r requirements.txt
```

Step 2 — Run the full pipeline

```
python setup.py
```

This single command executes the entire pipeline end to end: dataset generation → cleaning → feature engineering → 15 EDA charts → training of all 5 models → best-model selection. On a typical machine, this completes in approximately 2 minutes.

Step 3 — Launch the dashboard

```
streamlit run dashboard/app.py
```

The application opens automatically in your default browser at <http://localhost:8501>.

10.3 Expected Console Output

A successful run of `setup.py` prints a structured progress log for each of the four stages, concluding with a summary confirming the best model and its AUC-ROC score, e.g. Best Model: Logistic Regression, AUC-ROC: 0.6978, with total pipeline runtime of roughly 130 seconds.

10.4 Re-running Individual Stages

Each stage can also be run independently for development or debugging purposes:

Stage	Command
Data generation only	<code>python -m data.generator</code>
Cleaning pipeline only	<code>python -m preprocessing.pipeline</code>
EDA charts only	<code>python -m eda.visualizations</code>

Stage	Command
Model training only	<code>python -m models.training</code>

11. Maintenance & Extension Guide

11.1 Using a Real Dataset

To replace the synthetic data with a real dataset, provide a CSV with the same 23 raw columns described in Section 3.2, save it as `data/job_placement_raw.csv`, and either skip the generation step or modify `data/generator.py` to load an external file instead of simulating one. No changes are needed downstream — the cleaning, feature engineering, EDA, and training stages operate on column names, not data provenance.

11.2 Adding a New Model

To add a new algorithm to the comparison in `models/training.py`:

11. Import the estimator class at the top of the file.
12. Add a new entry to the dictionary returned by `_define_models()`, following the existing `Pipeline(...)` or `bare-estimator` pattern.
13. Re-run `python setup.py` — the new model will automatically be included in every evaluation chart, the comparison table, and the best-model selection logic, with no other code changes required.

11.3 Adding a New Feature

To engineer an additional feature, add the derivation logic to `features/engineering.py` inside `add_features()`, and — if it should be used by the models — add its (encoded) column name to the `FEATURE_COLS` list in `models/training.py`. Remember to also update the Predict page's inline feature-construction logic in `dashboard/app.py` if the new feature should be predictable from user input.

11.4 Retraining Cadence

Because the dataset is synthetic, retraining is only meaningful after regenerating data or swapping in a real dataset. In a production deployment with a live data source, the recommended pattern is to schedule `setup.py` to run on a periodic basis (e.g., weekly) via a workflow scheduler, so the dashboard always reflects a freshly trained best model.

11.5 Known Limitations

- The dataset is synthetic; absolute performance numbers ($AUC \approx 0.70$) should be treated as a methodology baseline, not a claim about real-world predictive power.
- Categorical standardization uses fixed canonical dictionaries (Section 4.2) — a genuinely novel label variant not present in the mapping will not be corrected automatically.
- The Predict page currently defaults several inputs (e.g., `num_interviews`, `age`, `gender`) to fixed placeholder values rather than exposing every raw column as a form field, to keep the UI concise.

12. Appendix

12.1 Glossary

Term	Definition
AUC-ROC	Area Under the Receiver Operating Characteristic Curve — measures a classifier's ability to rank positive cases above negative cases across all thresholds; 0.5 = random, 1.0 = perfect.
Stratified K-Fold	A cross-validation method that preserves the class balance in every fold, ensuring fair evaluation on imbalanced targets.
Precision	Of all candidates predicted Placed, the fraction that were actually Placed.
Recall	Of all candidates actually Placed, the fraction the model correctly identified.
F1 Score	The harmonic mean of precision and recall — balances both concerns into one number.
Label Encoding	Converting categorical text values into integer codes so they can be used by ML algorithms.
StandardScaler	Transforms numeric features to have mean 0 and standard deviation 1, required for distance-based and linear models to perform well.

12.2 File Reference — Outputs Directory

Path	Contents
outputs/job_placement_cleaned.csv	Final cleaned + engineered dataset (50,000 x 39)
outputs/scaler.pkl	Fitted StandardScaler for numeric features
outputs/encoders.pkl	Fitted LabelEncoders for categorical features
outputs/eda_charts/01-15	Exploratory data analysis charts
outputs/eda_charts/16-20	Model evaluation charts
outputs/model_plots/*.pkl	One serialized model per algorithm, plus best_model.pkl
outputs/model_plots/feature_names.pkl	Ordered list of 25 features expected by the models
outputs/reports/cleaning_report.txt	Machine-readable cleaning pipeline summary
outputs/reports/model_evaluation.txt	Full per-model classification reports